Report: Unconstrained Optimization

Carlo Coletta 294507
Riccardo Vittimberga 344810

February 2025

Abstract

We chose to solve the assignment about Unconstrained Optimization methods, specifically we implemented algorithms to solve various problems using the Nelder Mead method and the modified Newton method. We picked the problems 1,16 and 25 from the suggested list to have different types of functions to test our methods. We run the experiment initially on the vector suggested by the problem to later evaluate the methods on 10 more perturbated vectors. Ultimately, we provided comparisons between various evaluation metrics on the average for the different starting vectors, followed by comments on the results.

1 Introduction to the methods

1.1 Nelder Mead method

The Nelder Mead method optimizes the starting function without using derivatives, but is a simplex type method. A simplex is a convex field of n+1 points in an n-dimensional space: we perform a series of operations on it and then evaluate the function at its vertices. We start by building a non-singular simplex and at every step we order the points in an ascending way depending on their function evaluation. Not having a starting simplex provided, we built it adding a fixed perturbation to every vertex. The first operation we do is the Reflection. We compute the barycenter of the best n points, cutting out the worst: we will compute a reflected point xR of the latter respect to the barycenter with given scaling $\rho > 0$ (we set it at 1). We then evaluate $f(xR)$: if $f(x1) <= f(xR) < f(xN)$ we will accept xR as a new point for the simplex (replacing the $n+1$ point) and then move to the next iteraion. That will mean we have a good amount of reduction, but not exceptional. If the prevoius condition is not satisfied we check if $f(xR) < f(x1)$: if so we move to our expansion phase, meaning we had an exceptional reduction. We compute a new point xE, expanding the simplex in the xR direction with a given magnitude χ, opposing the barycenter. If $f(xE) < f(xR)$ we will take xE as the new $n+1$ point, otherwise it will be xR. If $f(xR) >= f(xn)$ instead, it means we will have moderate or no reduction and proceed with the Contraction phase. We will compute the contraction point xC between the barycenter and the best among xR and $x(n+1)$ with parameter γ: if $f(xC) < f(x(n+1)$ we will accept xC as the new point, otherwise we will ultimately go to the shrinking phase. The shrinking phase consists of "reducing" the $n+1$ points in the best point direction with magnitude σ: that is the worst case scenario, as it means we have not improved our function.

1.2 Modified Newton method

The Newton method, instead relies on the computation of the gradient and Hessian of a function to move towards descend directions to reach the minimum at each iteration. The modified Newton method is an improvement of the Newton method, it in fact solves the issue of having a Hessian not positive definite: we will slightly modify it to ensure this property. For modifying the Hessian we adopted two different strategies. The first one was based on adding a new matrix, as small as possible, to the Hessian until it became positive definite. The problem of the new strategy is related to the cost of computing the smallest eigenvalue of the Hessian, which is necessary to build the new matrix. The other strategy is based on Cholesky factorization. If this factorization works properly, we can solve the linear system to retrieve descent direction. The main drawback of using Cholesky is that the algorithm can become really expensive with many iterations. Furthermore, we will need an efficient line search among the directions at each iteration: to do so we use a backtracking strategy satisfying the Armijo condition. We implemented

a version using exact derivatives, a version using derivatives computed with finite differences, with both fixed and variable steps.

1.3 Rate Of convergence

The rate of convergence in numerical methods is used to measure how quickly a sequence of approximations reaches the exact solution. A higher rate indicates faster convergence. The experimental rate of convergence (ROC) is an empirical measure of how the error decreases between successive approximations:

$$p = \frac{\log\left(\frac{|e_{n+1}|}{|e_n|}\right)}{\log\left(\frac{|e_n|}{|e_{n-1}|}\right)}$$

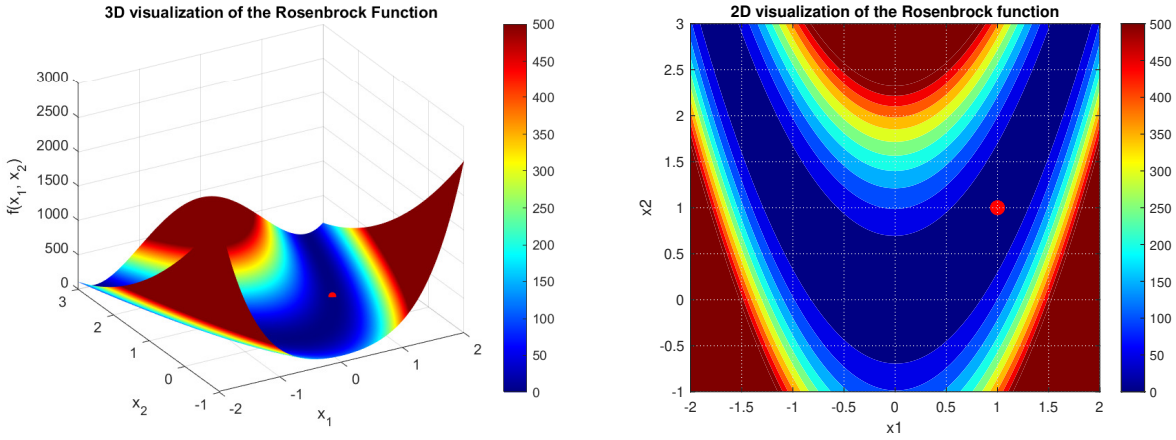
where e_n is the error at step n. The choice of this specific method relies on the fact that we do not know the global minima, so this method results helpful, since it does not consider them during computations.

2 Methods evaluation on the Rosenbrock function

2.1 Problem description

We tested our initial implementations on the Rosenbrock function for x $\in R^2$:

$$f(x) = 100(x_2 - x_1^2)^2 + (1 - x_1)^2$$



As we can see from the plots and expect analyzing the function, all its values are positive except for its global minimum 0, which is located in (1,1), considering we are in a 2-dimensional space.

We will use 2 different starting points:

$$x_a^{(0)} = (1.2, 1.2)$$
$$x_b^{(0)} = (-1.2, 1)$$

2.2 Nelder mead method

For the Nelder mead method we used the standard parameters: $\rho = 1$, $\chi = 2$, $\gamma = 0.5$, $\sigma = 0.5$. with a maximum of $kmax = 5000$ iterations and a stopping tolerance of $tol = 10^{-6}$.

	xbest		fbest	n_iter	exec_time	roc
x_a^(0)	0.98996	0.9801	0.00010135	28	0.016553	Inf
x_b^(0)	1.0003	1.0007	1.3951e-07	94	0.0038625	Inf
Avg	0.99515	0.99038	5.0743e-05	61	0.010208	Inf

For both our starting points, the methods provide a great optimization of the function, with the final vector tending to (1,1) and its evaluation to 0. The rate of convergence tends to infinite: in fact, the best values likely remain the same getting closer to the last iterations, and this leads to a zero difference between the evaluation at a given point and the best experimental value, consequentially causing a 0 by 0 division.

2.3 Modified newton method

For the Modified Newton method we used the following parameters: $\rho = 0.5$, $=10^{-4}$, $btmax = 50$ (max number of iteration while backtracking), $\tau = 100$, with a maximum of $kmax = 5000$ iterations and a stopping tolerance of $tol = 10^{-6}$.

	xbest		fbest	grad_fk	n_iter	exec_time	roc
x_a^(0)	1	1	0.00010135	1.436e-11	28	0.015844	2.6617
x_b^(0)	1	1	1.3951e-07	4.4733e-10	94	0.002846	1.6345
Avg	1	1	5.0743e-05	2.3085e-10	61	0.0093451	2.1481

The newton method converges easily to the solution as well. In this case the rate of convergence is around 2.

2.4 Comments

The Modified Newton Method converges faster than the Nelder-Mead method for low-dimensional functions because in low dimensions, computing gradients and Hessians is not costly, whereas Nelder-Mead is a derivative-free method that relies on function evaluations and simplex transformations, leading to slower convergence. The results displayed of Newton are the ones retrieved with the version which uses the Cholesky factorization. We could have used the other version too, obtaining similar results.

3 Problem 1: Chained Rosenbrock function

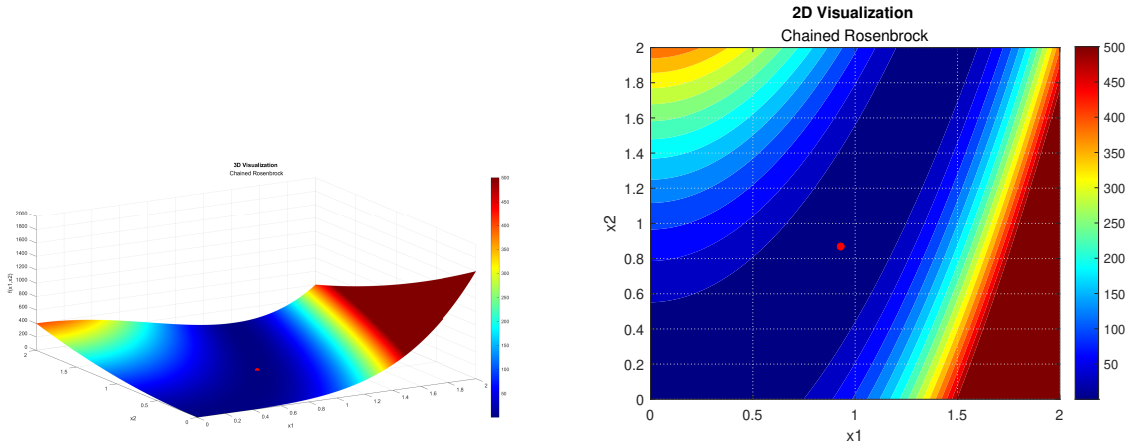
3.1 Problem description

As first problem, we chose the Chained Rosenbrock function to have a quick comparison with the standard version and see how it changes with bigger dimensions.

$$F(x) = \sum_{i=2}^{n} \left[100(x_{i-1}^2 - x_i)^2 + (x_{i-1} - 1)^2\right] \tag{1}$$

The suggested starting vector was:

$$x_i = \begin{cases} -1.2 & \text{mod}(i, 2) = 1 \\ 1.0 & \text{mod}(i, 2) = 0 \end{cases}$$



Example of the function in dimension n=2

Having the same structure as the previous Rosenbrock we can easily see that the function is made only of positive contributions and its global minimum is 0, reachable from a vector of ones.
We can also notice that the function has a pretty flat region in the neighborhood of the minimum and then skyrockets getting far. This will likely cause this function to be difficult to minimize. In order to compute the derivatives we had to look closely to the structure of the problem.

Gradient Computation

The gradient of $F(x)$, denoted as $\nabla F(x)$, has the following components:

$$g_1 = -400x_1(x_2 - x_1^2) + 2(x_1 - 1), \tag{2}$$
$$g_i = 200(x_i - x_{i-1}^2) - 400x_i(x_{i+1} - x_i^2) + 2(x_i - 1), \quad \text{for } 2 \le i \le n-1, \tag{3}$$
$$g_n = 200(x_n - x_{n-1}^2). \tag{4}$$

Hessian Computation

The Hessian matrix $H(x)$ is a tridiagonal matrix where:

- The diagonal elements are:

$$H_{ii} = 2 + 12 \cdot 100 \cdot x_i^2 - 4 \cdot 100 \cdot x_{i+1}, \quad \text{for } i = 1, \ldots, n-1. \tag{5}$$

$$H_{nn} = 200. \tag{6}$$

- The off-diagonal elements are:

$$H_{i,i+1} = H_{i+1,i} = -400x_i, \quad \text{for } i = 1, \ldots, n-1. \tag{7}$$

We then implemented the Hessian using a sparse tridiagonal symmetric structure for efficiency.

3.2 Nelder mead evaluation

For the Nelder mead method we used the standard parameters: $\rho = 1$, $\chi = 2$, $\gamma = 0.5$, $\sigma = 0.5$. with a maximum of $kmax = 5000$ iterations and a stopping tolerance of $tol = 10^{-6}$.
We run the evaluation for all the required dimensions 10,25,50. For each of them, we started our computations from the suggested starting vector and then proceeded to evaluate from 10 more vectors computed as a random perturbation with uniform distribution between [-1,1] applied to the starting vector. In the following table we reported the average for the most interesting outcomes, computed for each input dimension, specifically: the value of the minimum point found, the number of iterations needed, time execution time required, the rate of convergence and the number of failures.

	fbest	n_iter	exec_time	roc	n_fails
N=10	4.0992	2613.5	0.055579	Inf	0
N=25	64.884	5000	0.19726	Inf	11
N=50	260.84	5000	0.36928	Inf	11

Our algorithm struggles to give us great results. The evaluation of the function is pretty close to 0 for N=10, but then grows proportionally with the increasing number of dimensions. Also the performance of the algorithm is not optimal: it gets to the requested tolerance only for the first case and still needs a huge number of iterations. For N=25,50 the methods stops when reaching kmax. Still, the execution time of the algorithm is very fast: it computes the next state of the problem in little time, but the effectiveness of the improvement from the iteration is not great. Maybe incrementing the number of iterations would probably get to a better result. However the algorithm might be stuck in a local minimum or non-optimal region where the simplex keeps shrinking but does not escape. This can happen in flat regions of the function where function values change very slowly, preventing progress. If the function has steep ridges or very narrow valleys (e.g., Rosenbrock function), Nelder-Mead may struggle because it does not use gradient information. A possible solution could be looking for a better initialization, ensuring that the starting points are well spread in the search space.

3.3 Modified Newton evaluation with exact derivatives

We ran again all the 11 experiments in dimensions N=$10^3, 10^4, 10^5$. We used the following parameters: $\rho = 0.5$, $c1 = 1e-4.$, $btmax = 50$, $\tau = 100$, with a maximum of $kmax = 5000$ iterations and a stopping tolerance of $tol = 10^{-4}$. Those are the results of our Newton method evaluation:

	fbest	grad_norm	n_iter	exec_time	roc	n_fails
N=1000	2.5369	2.2132e-05	1338.6	0.12903	2.0864	0
N=10000	6172.1	12.017	5000	3.9696	1.6045	11
N=100000	95243	16.373	5000	62.013	1.2309	11

As we could expect from the plot of the function we had a tough time trying to optimize this function: only for N=10^3 we had an acceptable result, with the function evaluation heading towards 0 as well as the norm of the gradient, and reasonable execution time paired with experimental rate of convergence around 2. It fails for n=$10^4, 10^5$ because of the extreme ill-conditioning and complexity of the Chained Rosenbrock function. The function has narrow, curved zones with extreme variations in curvature,

creating regions with steep slopes and flat areas, making the optimization difficult. The Hessian matrix becomes ill-conditioned, leading to instability in Newton's method, which relies on solving

$$H\Delta x = -\nabla f$$

When the Hessian is near-singular, the method struggles to compute Δx accurately. For larger values of n, these issues are magnified, especially in high-dimensional spaces where the Hessian becomes increasingly ill-conditioned, making the descent direction hard to compute and leading to numerical errors. The Cholesky-based Modified Newton method fails because the Hessian needs to be positive definite, but as n grows larger, the Hessian becomes nearly singular. When n increases, the Hessian becomes unstable, leading to numerical issues. Additionally, the method becomes inefficient for large problems due to high computational cost and memory usage, even for a multiplying factor equal to 10 of the diagonal matrix added to the Hessian. The method that computes the smallest eigenvalue also fails because finding this eigenvalue in high dimensions is computationally expensive. Small errors in estimating the eigenvalue lead to poor updates to the Hessian, which results in ineffective descent directions and slow convergence. The results displayed come from the modified Newton method which exploits the Cholesky factorization because still was the one working better. We implemented few possible modifications such as preconditioning the Hessian and a deep parameter tuning, but despite these attempts, they did not resolve the issue fully.

3.4 Modified Newton evaluation with finite differences derivatives

Finite difference methods approximate derivatives using function evaluations at perturbed points. Given a function $F(x)$, its gradient and Hessian can be approximated as follows:

Gradient Approximation

The gradient is computed using the central difference formula:

$$\frac{\partial F}{\partial x_i} \approx \frac{F(x + he_i) - F(x - he_i)}{2h}, \tag{8}$$

where e_i is the unit vector in the i-th coordinate direction and h is the step size. The choice of h is crucial for accuracy and stability, and it can be either:

- A constant step size: $h =$ fixed value.
- A relative step size: $h = \alpha|x_i|$, where α is a small constant.

So we computed the gradient of the Chained Rosenbrock function using finite differences as:

$$\frac{\partial F}{\partial x_j} \approx \frac{1}{2h} \sum_{i=2}^{n} \left\{ 100 \left[\left((x_{i-1} + h)^2 - x_i^2 \right) - \left((x_{i-1} - h)^2 - x_i^2 \right) \right] + \left[\left((x_{i-1} + h) - 1 \right)^2 - \left((x_{i-1} - h) - 1 \right)^2 \right] \right\}.$$

With the unique special case of $i = 1$:

$$\frac{\partial f}{\partial x_1} \approx 400x_1(x_1^2 + h_1^2 - x_2) + 2(x_1 - 1), \quad \text{for } i = 1$$

Hessian Approximation

The Hessian matrix contains second-order derivatives and is approximated using:

$$H_{ij} \approx \frac{F(x + he_i + he_j) - F(x + he_i - he_j) - F(x - he_i + he_j) + F(x - he_i - he_j)}{4h^2}. \tag{9}$$

For $i = j$, this reduces to:

$$H_{ii} \approx \frac{F(x + he_i) - 2F(x) + F(x - he_i)}{h^2}. \tag{10}$$

First element: H_{11}

$$H_{1,1} \approx \frac{100(x_1 + 2h_1)^4 - 1600h_1^2 x_2 + 8h_1^2 - 200x_1^4 + 100(x_1 - 2h_1)^2}{h_1^2}$$

Element of H_{ii} for i=2,3,...n-1

$$H_{i,i} \approx \frac{800h_i - x_{i-1}^2 + 2x_{i-1} + x_i^2 - 2x_i + 100(x_i + 2h_i)^4 - 1600h_i^2 x_{i+1} + 8h_i^2 - 200x_i^4 + 100(x_i - 2h_i)^2}{h_i^2}$$

Last element: H_{nn}

$$H_{n,n} \approx \frac{800h_n^2 - x_{n-1}^2 + 2x_{n-1} + 2x_n^2 - 2x_n}{h_n^2}$$

$$H_{1,2} \approx \frac{-1600\, h_1\, h_2\, x_1}{4h_1^2}$$

$$H_{i-1,i} \approx \frac{-400\, x_{n-1}\, h_{n-1}}{h_n}$$

3.5 Modified Newton evaluation with finite differences

We computed the finite differences just for the main diagonal and the upper main diagonal. We started from the generic formula and then we noticed that developing many terms a lot of them cross out giving back reasonable values. We also notice that h does not have high values of exponents so numerical cancellation should not be high. Even though we paid attention to computations the method does not work for some mistakes we could not find in our approximated Hessian. When running the code, in fact, we had an access problem into our rate of convergence vector, most likely meaning that our function does not reach an iteration number $k > 1$ (so the method stops at the first iteration) as requested to evaluate the convergence rate. We then decided to proceed with a general function for computing the Hessian matrix. As we thought this function did not help us since we are working with large scale problems and it takes too much time to use general criteria.

	fbest	grad_norm	n_iter	exec_time	roc	n_fails
h=1.000000e-02	5.3249e+05	1.9185e-05	45.273	0.66784	1.9064	0
h=1.000000e-04	5.2791e+05	0.00039324	49	0.74232	Inf	1
h=1.000000e-06	5.2828e+05	0.00075301	96.364	1.431	Inf	2
h=1.000000e-08	5.2474e+05	0.18407	151.45	2.1477	Inf	3
h=1.000000e-10	5.2502e+05	2906.6	389.64	5.5305	0.91581	8
h=1.000000e-12	5.2934e+05	5020.1	179.91	2.5854	Inf	3

4 Problem 16: Banded trigonometric problem

4.1 Problem description

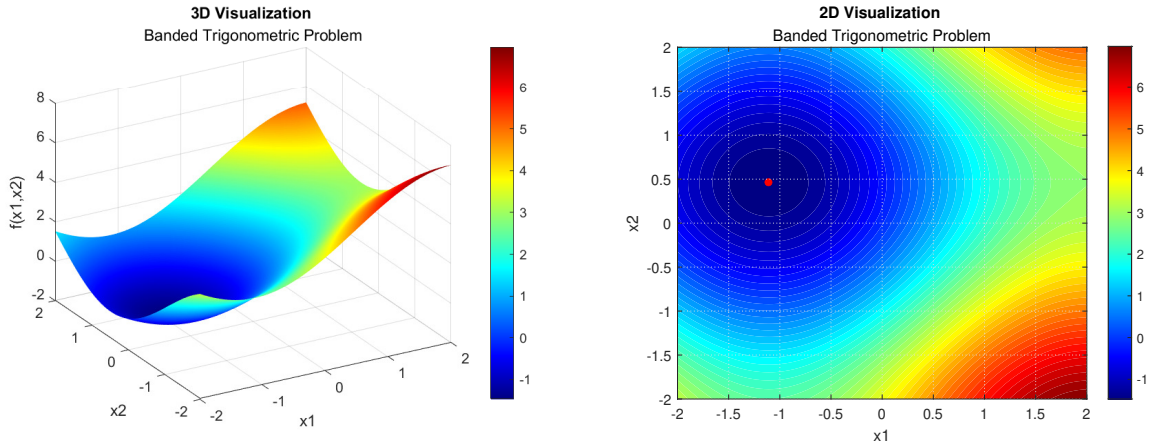
We then moved to a trigonometric equation, adding weighted and cyclic contributions to itself.

$$F(x) = \sum_{i=1}^{n} i\left[(1 - \cos x_i) + \sin x_{i-1} - \sin x_{i+1}\right], \quad x_0 = x_{n+1} = 0 \tag{11}$$

The suggested starting vector was a vector of ones of dimension n (except for the marginal cases).

$$\bar{x}_i = 1,\ i \geq 1.$$

The following are the 3D and 2D plots of the function.



Example of the function in dimension n=2

The presence of trigonometric terms, particularly sine and cosine, introduces nonlinearity into the function, resulting in multiple local minima. The combination of sinusoidal components and index-dependent coefficients complicates finding a global minimum.

Gradient and Hessian Computation

The gradient of $F(x)$ is obtained by differentiating each term with respect to x_i. The components of the gradient are:

$$\frac{\partial F}{\partial x_1} = 2\cos x_1 + \sin x_1, \tag{12}$$

$$\frac{\partial F}{\partial x_i} = i\sin x_i + 2\cos x_i, \quad \text{for } 2 \leq i \leq n-1, \tag{13}$$

$$\frac{\partial F}{\partial x_n} = n\sin x_n - (n-1)\cos x_n. \tag{14}$$

These terms include sine and cosine components that vary depending on the index i, making the gradient nonlinear.

The Hessian matrix, which consists of second derivatives, is a diagonal matrix due to the nature of the function. The second derivatives are computed as follows:

$$H_{11} = -2\sin x_1 + \cos x_1, \tag{15}$$

$$H_{ii} = i\cos x_i - 2\sin x_i, \quad \text{for } 2 \leq i \leq n-1, \tag{16}$$

$$H_{nn} = n\cos x_n + (n-1)\sin x_n. \tag{17}$$

Since the Hessian matrix is diagonal, it has a banded structure, making it computationally efficient for numerical methods that exploit sparsity.

4.2 Nelder mead evaluation

For the Nelder mead method we used the standard parameters: $\rho = 1.2$, $\chi = 2.2$, $\gamma = 0.5$, $\sigma = 0.5$. with a maximum of $kmax = 5000$ iterations and a stopping tolerance of $tol = 10^{-4}$.

	fbest	n_iter	exec_time	roc	n_fails
N=10	-2.8097	872.91	0.053864	Inf	0
N=25	10.989	4241.1	0.51756	Inf	2
N=50	109.95	5000	1.4087	Inf	11

In this case Nelder Mead works properly just for 10 dimension. Even if it is impossible knowing the actual minimum, for this function we know that depends on the magnitude of the index and we can assume greater is the index, more negative is the function. Anyway as the number of dimensions increases, the simplex has a harder time exploring the search space effectively and converging to the optimal solution, especially for complex or oscillatory objective functions. The function landscape in higher dimensions becomes more difficult for the algorithm to navigate, leading to slower convergence or getting stuck in local minima, which can be really common in trigonometric functions. Additionally, the method lacks of gradient information, which would help the simplex navigate such space in higher dimensions.

4.3 Modified Newton evaluation with exact derivatives

Like previously, we run the 11 experiments in dimensions N=$10^3, 10^4, 10^5$. We used the following parameters: $\rho = 0.5$, $c1 = 1e-4.$, $btmax = 50$, $\tau = 100$, with a maximum of $kmax = 5000$ iterations and a stopping tolerance of $tol = 10^{-4}$.

	fbest	grad_norm	n_iter	exec_time	roc	n_fails
N=1000	-427.4	2.8716e-06	15.909	0.0082712	3.8027	0
N=10000	-4159.9	1.1647e-05	19.636	0.032638	5.4822	0
N=100000	-41444	0.00042613	4546	60.832	1.3274	0

We could not clearly see what the minimum of the function could be from our graphical and algebraic analysis of the equation. Supposing our algorithm works correctly, we can imagine it stretches to a negative number proportional to the dimension of the problem. This does make sense since the indexes which multiply function grow as the problem does. Our method can find pretty easily the minimum of the function for N=$10^3, 10^4$, with the norm of the gradient asymptotically going to 0, a reduced number of iterations, very fast execution time and superlinear rate of convergence. For N=10^5 instead, even though the method never gets to the maximum number of iterations allowed, leading to a failure, it gets very close and requires a considerable amount of time (about a minute on average) to optimize the function.

4.4 Modified Newton evaluation with finite differences derivatives

We approximate the gradient using the centered finite difference formula:

$$\frac{df}{dx_i} \approx \frac{f(x_i + h_i) - f(x_i - h_i)}{2h_i} \tag{18}$$

where h_i is the step size. Given the function we get:

$$\frac{f(x_i + h_i) - f(x_i - h_i)}{2h_i} = \frac{2i \sin x_i \sin h_i + 4 \cos x_i \sin h_i}{2h_i}. \tag{19}$$

Thus, the final formula for $0 < i < n$ is:

$$\nabla f_i \approx \frac{2i\sin(x_i)\sin(h_i) + 4\cos(x_i)\sin(h_i)}{2h_i}. \tag{20}$$

Instead for the terminal case:

$$\nabla f_n = \frac{2n\sin(x_n)\sin(h_n) - 2(n-1)\cos(x_n)\sin(h_n)}{2h_n} \tag{21}$$

Hessian Approximation

The Hessian is computed using:

$$\frac{\partial^2 f}{\partial x_i^2} \approx \frac{f(x_i + h_i) - 2f(x_i) + f(x_i - h_i)}{h_i^2}. \tag{22}$$

Then thanks to the use of addition formulas of sine and cosine, plus the usage of Taylor expansion series we get as Hessian computed with centered finite differences:

$$\frac{(i\cos x_i - 2\sin x_i) - h_i(i\sin x_i + 2\cos x_i)}{h_i^2}. \tag{23}$$

Thus, the final Hessian formula is:

$$H_{ii} \approx i\cos x_i - 2\sin x_i - h_i(i\sin x_i + 2\cos x_i). \tag{24}$$

	fbest	grad_norm	n_iter	exec_time	roc	n_fails
h=1.000000e-02	-427.39	1.3418	492.64	0.50789	1.0247	10
h=1.000000e-04	-427.38	1.1283	500	0.50615	1.0128	11
h=1.000000e-06	-427.39	1.3808	500	0.4989	0.994	11
h=1.000000e-08	-427.38	1.5098	500	0.49392	0.99956	11
h=1.000000e-10	-427.39	1.1425	500	0.50827	1.005	11
h=1.000000e-12	-427.39	1.4722	498.27	0.50539	0.99772	9

Function behavior for dimension N=10^3

	fbest	grad_norm	n_iter	exec_time	roc	n_fails
h=1.000000e-02	-4157.6	30.195	500	10.797	1.0246	11
h=1.000000e-04	-4157.1	34.774	500	10.837	0.98153	11
h=1.000000e-06	-4157.1	39.596	500	10.955	1.0003	11
h=1.000000e-08	-4157.1	36.037	500	11.164	1.0277	11
h=1.000000e-10	-4157.1	34.271	500	11.206	0.99335	11
h=1.000000e-12	-4157	31.635	500	10.979	1.0054	11

Function behavior for dimension N=10^4

The results show that finite differences find the same minima as the exact derivatives, confirming their accuracy in this context. For a step size k, they perform efficiently for both n=10^3 and n=10^4. For n=10^5 instead the computation took an average of 10 minutes per iteration, making it impossible to print the whole table. However, we computed the newton method for just the starting vectors and for fewer step sizes, and we ensured that the process led to a correctly approximated function, making it just an issue of efficiency.

```
f(xk): -41442.1184
norm gradf(xk): 24.4949
 N. of Iterations : 500/500
Execution time: 621.0561 seconds
Roc: 0.75843
```

Ex: results for h=10^{-2}

The optimization process was limited to 500 iterations because, despite the function converging within a few iterations, the stopping criterion based on the gradient norm was not met. After a certain point, the function value no longer improved, indicating that further iterations were unnecessary. The Taylor series expansion used in the finite difference approximation behaved as expected. For sufficiently small step sizes h, the approximation remained stable and provided accurate results. However, for larger values of h, in relative step-size adjustments, the method failed, which is a known limitation due to truncation and numerical errors. This behavior aligns with theoretical expectations, as finite differences are sensitive to the choice of h. Too small a value increases floating-point errors, while too large a value leads to poor approximations. Overall, the method worked efficiently, balancing computational speed and accuracy, making it a practical alternative to exact derivatives in this setting.

4.5 Modified Newton evaluation with finite differences derivatives using variable steps

	fbest	grad_norm	n_iter	exec_time	roc	n_fails
h=1.000000e-02	10225	964.59	465.36	0.61366	0.99457	0
h=1.000000e-04	1323.6	100.04	455.64	0.59279	0.95818	0
h=1.000000e-06	-427.39	1.0514	495.09	0.52817	0.99655	0
h=1.000000e-08	-427.4	0.63145	500	0.52423	1.0095	0
h=1.000000e-10	-427.39	0.92212	500	0.50263	1.0091	0
h=1.000000e-12	-427.38	1.629	500	0.51141	1.0001	0

Function behavior for dimension N=10^3

	fbest	grad_norm	n_iter	exec_time	roc	n_fails
h=1.000000e-02	1.6626e+06	61354	500	10.377	-Inf	0
h=1.000000e-04	30401	170.82	500	9.6409	0.98053	0
h=1.000000e-06	10490	219.13	428.64	8.3135	1.018	0
h=1.000000e-08	-4157.5	34.543	500	9.554	0.98281	0
h=1.000000e-10	-4157.3	37.55	500	9.6792	1.0061	0
h=1.000000e-12	-4157.3	37.367	500	9.8757	1.0001	0

Function behavior for dimension N=10^4

Using variable steps gives us not converging functions for $h = 10^{-2}, 10^{-4}$ for dimension $n = 10^3$ and for $h = 10^{-2}, 10^{-4}, 10^{-6}$ for dimension $n = 10^4$. For the other steps the results are similar to those computed with fixed steps. We encountered the same problem for $n = 10^5$. The image on the top corner page is representative. It does not convey any useful information for comparisons and analysis but we decide to put it there because we wanted to know that the algorithm works and converges. As a matter of fact the algorithm, even if it is slower respect to the one which uses exact derivatives but gives back the same results for h fixed. For variable h we get results similar to the $n = 10^4$. We estimated a total time of more than 10 hours for a whole run.

5 Problema 25: Extended Rosenbrock function

5.1 Problem description

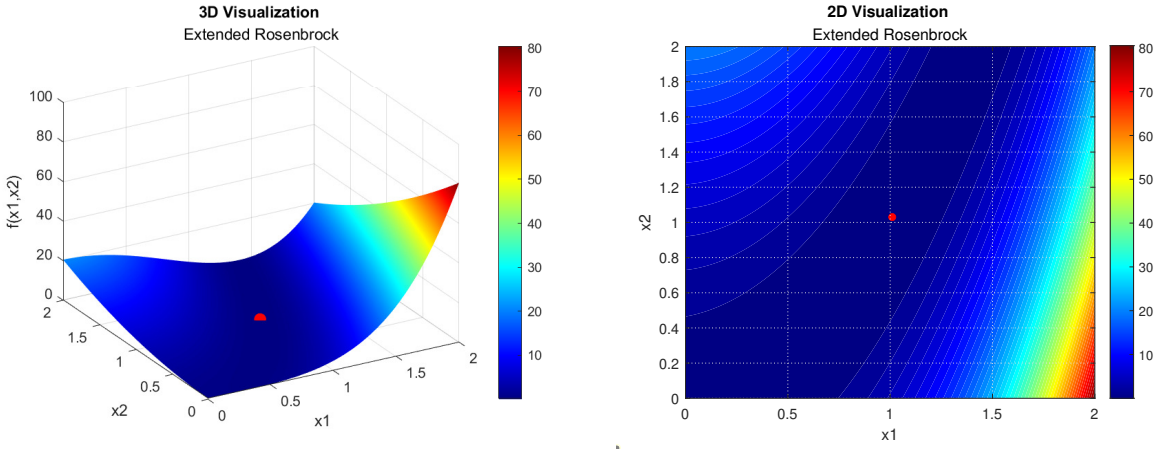
The problem we chose next is the Extended Rosenbrock function:

$$
\begin{aligned}
F(x) &= \frac{1}{2}\sum_{k=1}^{n} f_k^2(x), \\
f_k(x) &= \begin{cases} 10(x_k^2 - x_{k+1}), & \mathrm{mod}(k,2) = 1 \\ x_{k-1} - 1, & \mathrm{mod}(k,2) = 0 \end{cases}
\end{aligned}
\tag{25}
$$

The suggested starting vector is built as follows:

$$
x_l = \begin{cases} -1.2 & \mathrm{mod}(l,2) = 1 \\ 1.0 & \mathrm{mod}(l,2) = 0 \end{cases}
$$

We can visualize the function behavior in the following plots



Example of the function in dimension n=2

We can notice that the global minimum of the function is 0, when evaluated in (1,1). We do also see the presence of a flat region in the minimum neighborhood: this will likely cause a slower convergence in its proximity and might even cause the stagnation of the algorithm, especially using a zero-order method like the Nelder Mead.

Gradient and Hessian Computation

In this case we have to analyze our function depending on whether n is an even or odd number: in the latter we will have an $x(n+1)$ component, that we assume to be equal to $x1$.

Having an odd dimension, we will need to adjust the function to handle an extra component, that we decided to "include" as the first one as explained above.

$$
\frac{\partial F}{\partial x_1}(\mathbf{x}) = \frac{\partial}{\partial x_k}\left[\frac{1}{2}f_k^2(\mathbf{x}) + \frac{1}{2}f_{k+1}^2(\mathbf{x}) + \frac{1}{2}f_n^2(\mathbf{x})\right] = 200x_1(x_1^2 - x_2) + (x_1 - 1) - 100(x_n^2 - x_1)
$$

Instead for the general variable x_i:

$$\frac{\partial F}{\partial x_k}(\mathbf{x}) = \frac{\partial}{\partial x_k}\left[\frac{1}{2}f_{k-1}^2(\mathbf{x})\right] = -100(x_{k-1}^2 - x_k) \qquad \text{mod } (k,2) = 0$$
$$\frac{\partial F}{\partial x_k}(\mathbf{x}) = \frac{\partial}{\partial x_k}\left[\frac{1}{2}f_k^2(\mathbf{x}) + \frac{1}{2}f_{k+1}^2(\mathbf{x})\right] = 200x_k(x_k^2 - x_{k+1}) + (x_k - 1) \qquad \text{mod } (k,2) = 1$$

The latter equations will also be applied in the even n case.

The dimensionality of the problem will affect the Hessian computation as well. We will obtain a sparse matrix whose structure (the non-zero entries) will depend on whether n is even or odd.
For an even n:

$$\frac{\partial^2 F}{\partial x_k^2}(\mathbf{x}) = 100, \quad \frac{\partial^2 F}{\partial x_k \partial x_{k+1}}(\mathbf{x}) = 0, \quad \frac{\partial^2 F}{\partial x_k \partial x_{k-1}}(\mathbf{x}) = -200x_{k-1} \qquad \text{mod } (k,2) = 0$$
$$\frac{\partial^2 F}{\partial x_k^2}(\mathbf{x}) = 600x_k^2 - 200x_{k+1} + 1, \quad \frac{\partial^2 F}{\partial x_k \partial x_{k+1}}(\mathbf{x}) = -200x_k, \quad \frac{\partial^2 F}{\partial x_k \partial x_{k-1}}(\mathbf{x}) = 0 \qquad \text{mod } (k,2) = 1$$

We used the same strategy to handle the odd n case, so we need to take into account the f_n effect on x_1, with the external diagonal affected as well. We will then need the following adjustments:

$$\frac{\partial^2 F}{\partial x_1^2}(\mathbf{x}) = 600x_k^2 - 200x_{k+1} + 101$$
$$\frac{\partial^2 F}{\partial x_n \partial x_1}(\mathbf{x}) = \frac{\partial^2 F}{\partial x_1 \partial x_n}(\mathbf{x}) = -200x_n$$

5.2 Nelder mead evaluation

For the Nelder Mead evaluation we used the following parameters: $\rho = 1.5$, $\chi = 2.2$, $\gamma = 0.5$, $\sigma = 0.5$. with a maximum of $kmax = 5000$ iterations and a stopping tolerance of $tol = 10^{-4}$.

	fbest	n_iter	exec_time	roc	n_fails
N=10	4.895	596.27	0.015449	NaN	0
N=25	17.262	2339.5	0.095895	NaN	0
N=50	37.908	4843.6	0.38565	NaN	3

As we could expect, the Nelder method does not provide a great optimization of the function. Even if we respect our limit of iterations and the algorithm is very fast, the evaluation of the function in the last step is far from the expected value 0, and the value drastically increase when increasing the dimension of the problem. The stationary behavior of the function in the minimum neighborhood is the main reason of this poor performance and it affects the rate of converge computation as well: getting closer to the minimum will in fact lead to a 0 by 0 division.

5.3 Modified Newton evaluation with exact derivatives

Like previously, we run the 11 experiments in dimensions N=$10^3, 10^4, 10^5$. We used the following parameters: $\rho = 0.5$, $c1 = 1e-4.$, $btmax = 50$, $\tau = 100$, with a maximum of $kmax = 5000$ iterations and a stopping tolerance of $tol = 10^{-6}$.

	fbest	grad_norm	n_iter	exec_time	roc	n_fails
N=1000	5.6767e-16	5.1962e-08	26.455	0.017302	1.9326	0
N=10000	3.6226e-16	1.3699e-07	27.364	0.052802	1.9322	0
N=100000	2.528e-18	2.5764e-08	28.091	0.67404	2.2578	0

We can clearly see that, as we could expect from theory, the Newton method performs way better than the Nelder Mead: having information on gradient and hessian of the function allows a faster and more robust descent direction towards the optimal value of the function. We can see that ultimately the function always converges to 0, its expected global minimum, and the norm of its gradients behaves as well. The number of iterations is clearly reduced compared to the Nelder Mead method and the execution time is still very fast. Furthermore, the average results of the experiment are consistent across all dimensions and never it never gets even close to fail for its excessive number of iterations.

5.4 Modified Newton evaluation with finite differences derivatives

	fbest	grad_norm	n_iter	exec_time	roc	n_fails
h=1.000000e-02	0.096149	9.3814e-05	110.64	0.12822	1.0114	0
h=1.000000e-04	3.7406e-09	4.7217e-05	26.273	0.031533	1.6113	0
h=1.000000e-06	3.1708e-10	3.113e-05	25.636	0.033122	9.3129	0
h=1.000000e-08	4.3349e-11	3.0627e-05	26.182	0.031381	8.6007	0
h=1.000000e-10	4.4208e-11	1.4601e-05	26.364	0.031958	8.6913	0
h=1.000000e-12	2.04e-10	2.6403e-05	25.727	0.034139	8.8418	0

Function behavior for dimension N=10^3

	fbest	grad_norm	n_iter	exec_time	roc	n_fails
h=1.000000e-02	0.96127	9.3454e-05	121.27	1.3619	1.0114	0
h=1.000000e-04	1.0897e-08	1.7125e-05	27.909	0.32547	1.4873	0
h=1.000000e-06	6.501e-14	3.004e-06	26.818	0.31382	1.9192	0
h=1.000000e-08	4.3101e-10	1.0548e-05	26.818	0.3135	1.4316	0
h=1.000000e-10	4.2449e-10	1.0843e-05	26.727	0.31316	1.3848	0
h=1.000000e-12	4.2442e-10	1.0402e-05	26.818	0.31415	1.406	0

Function behavior for dimension N=10^4

	fbest	grad_norm	n_iter	exec_time	roc	n_fails
h=1.000000e-02	9.612	9.8363e-05	129.82	15.076	1.0115	0
h=1.000000e-04	1.0816e-07	5.6436e-05	28.545	3.4173	1.6371	0
h=1.000000e-06	1.5762e-13	2.7534e-06	26.909	3.2374	2.2071	0
h=1.000000e-08	4.4483e-14	3.429e-06	27.273	3.2661	2.2182	0
h=1.000000e-10	7.1134e-14	4.2605e-06	27.273	3.265	2.2309	0
h=1.000000e-12	3.3203e-15	1.9404e-06	27.273	3.2781	2.2011	0

Function behavior for dimension N=10^5

From these tables we can immediately notice how having a bigger step h (specifically for h=10^{-2}) makes the computation of the derivatives more expensive from both a computational and temporal point of view, and the function evaluation struggles to reach the expected target in the N=10^5 dimension. This behavior was completely predictable since nonlinearities in the function become more pronounced, leading to deviations from the true derivative. Additionally, larger h increases discretization errors, making the computed derivatives less reliable. The performances are pretty similar remaining in the same dimension, with the rate of convergence remaining more or less around the same values. This does not hold for N=10^3, as we see that reducing the step of the finite difference the rate of convergence becomes considerably greater. Comparing the results to the exact derivatives approach, we can see that the finite differences provide a slightly worse performance across the 3 different dimensions: the function evaluation does not converge to 0 with 6/7 orders of magnitude less, the norm of the gradient also is bigger of about 3 orders. The execution time is pretty similar regarding N=10^3, but then significantly increases for the higher dimensions. The rate of convergence varies depending on the selected h, but we can notice that it is slightly better for the exact derivatives, except for N=10^3, where a smaller step ensures a faster convergence of the function.

5.5 Modified Newton evaluation with finite differences derivatives using variable steps

	fbest	grad_norm	n_iter	exec_time	roc	n_fails
h=1.000000e-02	0.089219	9.4812e-05	103.09	0.13869	1.0121	0
h=1.000000e-04	2.6933e-09	4.0249e-05	26.364	0.034512	1.5983	0
h=1.000000e-06	1.3226e-10	1.9841e-05	26.091	0.032254	8.9037	0
h=1.000000e-08	1.1515e-10	1.8574e-05	26.091	0.031993	8.8197	0
h=1.000000e-10	4.7788e-11	2.0796e-05	26.273	0.032689	8.6346	0
h=1.000000e-12	4.2736e-11	3.0107e-05	26	0.033195	8.5615	0

Function behavior for dimension N=10^3

	fbest	grad_norm	n_iter	exec_time	roc	n_fails
h=1.000000e-02	0.89198	9.4576e-05	112.09	1.2646	1.0121	0
h=1.000000e-04	1.1061e-08	1.9137e-05	27.818	0.32621	1.4926	0
h=1.000000e-06	5.5532e-14	2.6343e-06	27	0.31875	1.9146	0
h=1.000000e-08	4.3079e-10	1.0622e-05	26.818	0.31732	1.4197	0
h=1.000000e-10	4.2449e-10	1.0759e-05	26.818	0.31613	1.4167	0
h=1.000000e-12	4.2442e-10	1.0566e-05	26.727	0.3152	1.4089	0

Function behavior for dimension N=10^4

	fbest	grad_norm	n_iter	exec_time	roc	n_fails
h=1.000000e-02	8.9191	9.2209e-05	138.45	16.095	1.0121	0
h=1.000000e-04	1.0795e-07	5.6081e-05	28.182	3.457	1.6393	0
h=1.000000e-06	2.3087e-13	3.7059e-06	26.909	3.3164	2.2219	0
h=1.000000e-08	6.3655e-14	3.794e-06	27.091	3.3404	2.2397	0
h=1.000000e-10	4.4853e-14	3.4249e-06	27.273	3.3596	2.2124	0
h=1.000000e-12	6.6832e-14	3.7264e-06	27.273	3.3464	2.214	0

Function behavior for dimension N=10^5

The evaluation with a variable step follows the same trends of the fixed step one.

6 Final thoughts

From our analysis, we can clearly see that the Modified Newton method performs significantly better than the Nelder-Mead method, even in higher-dimensional problems. As previously stated, this is due to the fact that Nelder-Mead lacks descent information from derivatives and Hessians.

We could expect better performance from simplex methods in lower-dimensional problems (fewer than 10 variables), as they avoid the computational cost of calculating gradients and Hessians. However, for smoother functions, the Newton method provides much faster convergence, especially as it approaches the minimum, theoretically achieving quadratic convergence.

It was also evident that the shape of the function impacts the efficiency of the optimization process. Exponential functions require significantly more iterations, and similar difficulties arise with functions that have flat regions near the optimum.

Main.m

```
clc
clear all

%The main is structured in different sections to be run modifing the
%arguments to adapt at every problem the requests

%Initial settings
random_seed = min(294507, 344810);
rng(random_seed);
dimensions_newton = [1e3, 1e4, 1e5];
dimensions_nelder = [10, 25, 50];

%Nelder parameters: we pass them as a vector
rho = 1.2;
chi = 2.2;
gamma = 0.5;
sigma = 0.5;
kmax = 5000;
tol = 1e-4;

parameters_nelder = [rho, chi, gamma, sigma, kmax, tol];


%Newton parameters: we pass them as a vector
rho = 0.5;
c1 = 1e-4;
btmax = 50;
tolgrad = 1e-4;
tau_kmax = 100;
kmax = 500;

parameters_newton = [rho, c1, btmax, tolgrad, tau_kmax, kmax];


%%
%FUNCTION PLOTTING
f = problema1();
generalFunctionPlot(f, [0,2], [0,2], 100, "Extended Rosenbrock");
```

```
39
40 %%
41 %ROSENBROCK FUNCTION
42 rosenbrockInitialCheck();
43
44 %%
45 %NELDER MEAD
46 wrapperNelder(dimensions_nelder, 1, parameters_nelder);
47
48 %%
49 %NEWTON WITH EXACT DERIVATIVES
50 wrapperNewtonExact(dimensions_newton, 1,
      parameters_newton);
51
52 %%
53 %NEWTON WITH FINITE DIFFERENCES AND FIXED STEP (FLAG 0)
54 for i=1:3
55     wrapperNewtonFinDiff(dimensions_newton(i), 1,
          parameters_newton, 0);
56 end
57
58
59 %%
60 %NEWTON WITH FINITE DIFFERENCES AND VARIABLE STEP (FLAG
      1)
61 for i=1:3
62     wrapperNewtonFinDiff(dimensions_newton(i), 1,
          parameters_newton, 1);
63 end
```

problema1.m

```
function [F,gradF,HessF] = problema1()

    F = @(x) sum(100 * (-x(2:end) + x(1:end-1).^2).^2 +
       (x(1:end-1) - 1).^2);
    gradF = @(x) compute_gradient(x);
    HessF = @(x) compute_hessian(x);


    function g = compute_gradient(x)
        n = length(x);
        g = zeros(n, 1);

        % Gradient for the first element
        g(1) = -400 * x(1) * (x(2) - x(1)^2) + 2 * (x(1)
            - 1);

        % Gradient for the interior elements
        for i = 2:n-1
            g(i) = 200 * (x(i) - x(i-1)^2) - 400 * x(i)
               * (x(i+1) - x(i)^2) + 2 * (x(i) - 1);
        end

        % Gradient for the last element
        g(n) = 200 * (x(n) - x(n-1)^2);
    end

   function Hessf = compute_hessian(x)
        n = length(x);
        diags = zeros(n, 3);  % Matrix to store diagonal
            elements of Hessian

        % Diagonal and off-diagonal elements for the
           first row
        diags(1, 1) = 2 + 12*100*x(1)^2 - 4*100*x(2);
        diags(1, 2) = -4*100*x(1);

        % Diagonal and off-diagonal elements for the
           last row
        diags(n, 1) = 2*100;
        diags(n, 2) = -4*100*x(n-1);

```

```
36          % Diagonal and off-diagonal elements for the
               second to last row
37          diags(n-1, 3) = -4*100*x(n-1);
38
39          % Loop to fill in the Hessian for interior
               points
40          for k = 2:n-1
41              diags(k, 1) = 2*100 + 12*100*x(k)^2 - 4*100*
                   x(k+1) + 2;
42              diags(k, 2) = -4*100*x(k-1);
43              diags(k, 3) = -4*100*x(k);
44          end
45
46          % Return the sparse Hessian matrix
47          Hessf = spdiags(diags, [-1, 0, 1], n, n);
48      end
49
50
51
52  end
```

problema16.m

```
function [F, grad, Hessian] = problema16()


    F = @(x) sum((1:length(x))' .* ((1 - cos(x)) + [0;
       sin(x(1:end-1))] - [sin(x(2:end)); 0]));
    grad = @(x) computeGradient(x);
    Hessian = @(x) computeHessian(x);
end


function grad_val = computeGradient(x)
    n = length(x);
    grad_val=zeros(n,1);
    grad_val(1)=2*cos(x(1))+sin(x(1));
    for i=2:n-1
        grad_val(i)=i*sin(x(i))+2*cos(x(i));
    end
    grad_val(n)=n*sin(x(n))-(n-1)*cos(x(n));
end


function hess_val = computeHessian(x)
    n = length(x);


    main_diag = zeros(n, 1);
    main_diag(1) = -2*sin(x(1)) + cos(x(1));

    for i = 2:n-1
        main_diag(i) = i*cos(x(i))-2*sin(x(i));
    end

    main_diag(n) = n*cos(x(n)) + (n-1)*sin(x(n));

    hess_val = spdiags(main_diag, 0, n, n);
end
```

problema25.m

```
function [F, gradF, HessF] = problema25()


    F = @(x) function_pb25(x);
    gradF = @(x) grad_pb25(x);
    HessF = @(x) hessian_pb25(x);
end


function val = function_pb25(x)
    n = length(x);
    val = 0;

    for k = 1:2:n-1
        val = val + 10 * (x(k)^2 - x(k+1))^2;
    end
    for k = 2:2:n-1
        val = val + (x(k-1) - 1)^2;
    end

    if mod(n,2) == 1
        val = val + (10*x(n)^2 - x(1))^2;
    else
        val = val + (x(n-1) - 1)^2;
    end

    val = 0.5 * val;
end


function grad = grad_pb25(x)
    n = length(x);
    grad = zeros(n,1);

    if mod(n,2) == 0
        grad(1:2:n-1) = 200*x(1:2:n-1).^3 - 200*x(1:2:n
           -1).*x(2:2:n) + x(1:2:n-1) - 1;
        grad(2:2:n) = -100*(x(1:2:n-1).^2 - x(2:2:n));
    else
        grad(1) = 200*x(1)^3 - 200*x(1)*x(2) + x(1) - 1
           - 100*(x(n)^2 - x(1));
```

```
        grad(3:2:n-1) = 200*x(3:2:n-1).^3 - 200*x(3:2:n
            -1).*x(4:2:n) + x(3:2:n-1) - 1;
        grad(2:2:n-2) = -100*(x(1:2:n-3).^2 - x(2:2:n-2)
            );
        grad(n) = 200*x(n).^3 - 200*x(n)*x(1) + x(n) -
            1;
    end
end


function val = hessian_pb25(x)
    n = length(x);
    diags = zeros(n,5);


    if mod(n,2) == 0
        diags(2:2:n,1) = 100;
        diags(1:2:n-1,1) = 600*x(1:2:n-1).^2 - 200*x
            (2:2:n) + 1;
    else
        diags(1,1) = 600*x(1)^2 - 200*x(2) + 101;
        diags(2:2:n,1) = 100;
        diags(3:2:n-1,1) = 600*x(3:2:n-1).^2 - 200*x
            (4:2:n) + 1;
        diags(n,1) = 600*x(n).^2 - 200*x(1) + 1;
    end


    diags(1:2:n-1,3) = -200*x(1:2:n-1);
    diags(2:2:n-2,3) = 0;

    l
    diags(3:2:n-1,2) = 0;
    diags(2:2:n,2) = -200*x(1:2:n-1);

    if mod(n,2) == 1
        diags(1,5) = -200*x(n);
        diags(n,4) = -200*x(n);
    end

    val = spdiags(diags, [0,1,-1, n-1, -(n-1)], n, n);
end
```

generalFunctionPlot.m

```
function generalFunctionPlot(f, x_range, y_range,
   resolution, name)

    %Desired point range
    x1 = linspace(x_range(1), x_range(2), resolution);
    x2 = linspace(y_range(1), y_range(2), resolution);

    [X1, X2] = meshgrid(x1, x2);

    Z = arrayfun(@(a, b) f([a; b]), X1, X2);

    [~, minIndex] = min(Z(:));
    [row, col] = ind2sub(size(Z), minIndex);
    x_min = X1(row, col);
    y_min = X2(row, col);
    f_min = Z(row, col);

    %Print 3d
    figure;
    surf(X1, X2, Z, 'EdgeColor', 'none');
    shading interp;
    colormap jet;
    colorbar;
    clim([min(Z(:)), min(500, max(Z(:)))]);
    hold on;
    plot3(x_min, y_min, f_min, 'ro', 'MarkerSize', 8, '
       MarkerFaceColor', 'r');

    xlabel('x1');
    ylabel('x2');
    zlabel('f(x1,x2)');
    title('3D Visualization ', name);
    view([-30, 30]);
    grid on;

    %Print 2d (top view)
    figure;
    contourf(X1, X2, Z, 50, 'LineColor', 'none');
    colormap jet;
    colorbar;
    clim([min(Z(:)), min(500, max(Z(:)))]);
```

```
    hold on;
    plot(x_min, y_min, 'ro', 'MarkerSize', 5, '
       MarkerFaceColor', 'r');

    xlabel('x1');
    ylabel('x2');
    title('2D Visualization ', name);
    grid on;

end
```

generateHypercubePoints.m

```
function new_points = generate_hypercube_points(x,
   n_vectors)
    n = length(x);
    new_points = (2 * rand(n, n_vectors) - 1) + x(:);
end
```

rosenbrockFunction.m

```
function [f,g,h] = rosenbrockFunction()

    f = @(x) sum(100 * (-x(2:end) + x(1:end-1).^2).^2 +
       (x(1:end-1) - 1).^2);
    g = @(x) compute_gradient(x);
    h = @(x) compute_hessian(x);

    function gradf = compute_gradient(x)
        n = length(x);
        gradf = zeros(n,1);

        for k = 2:n-1
            gradf(k,1) = -2*100*(x(k-1)^2 - x(k)) + 2*(x
               (k) -1) +4*100*x(k)*(x(k)^2 - x(k+1));
        end

        gradf(1,1) = 2*(x(1) -1) + 4*100*x(1)*(x(1)^2 -
           x(2));
        gradf(n,1) = -2*100*(x(n-1)^2 - x(n)) ;

    end

    function Hessf = compute_hessian(x)
        n = length(x);
        diags = zeros(n,3);

        diags(1,1) = 2 + 12*100*x(1)^2 - 4*100*x(2);
        diags(n,1) = 2*100;
        diags(n-1,3) = -4*100*x(n-1);
        diags(n,2) = -4*100*x(n-1);

        for k = 2:n-1
           diags(k,1) = 2*100 + 12*100*x(k)^2 - 4*100*x(
              k+1) +2;
           diags(k-1,3) = -4*100*x(k-1);
           diags(k,2)= -4*100*x(k-1);
        end


        Hessf = spdiags(diags, [0, +1, -1], n, n);

```

```
38      end
39
40
41  end
```

rosenbrockInitialCheck.m

```
function rosenbrockInitialCheck()
    x0_a = [1.2, 1.2]';
    x0_b = [-1.2, 1]';
    [f,g,h] = rosenbrockFunction();

    generalFunctionPlot(f, [-2,2], [-1,3], 100, "
       Rosenbrock");

    %Nelder mead parameters
    rho = 1;
    chi = 2;
    gamma = 0.5;
    sigma = 0.5;
    kmax = 5000;
    tol = 1e-6;

    %NELDER MEAD
    %NB: pass x0 as a column vector, implement how to
       handle in

    disp('****Nelder mead analysis:****');

    t1_a = tic;
    [xbest_a, iter_a, fbest_a, roc_a] = nelderMead(f,
       x0_a, rho, chi, gamma, sigma, kmax, tol,0);
    time_a = toc(t1_a);

    xbest_a = xbest_a';
    norm_gradf_a = norm(g(xbest_a));
    roc_a = roc_a(end);

    disp('****Results for vector x0_a:****\n');
    disp('Final x:'), disp(xbest_a);
    disp(['Norma di gradf(xk): ', num2str(norm_gradf_a)
       ]);
    disp(['f(xk): ', num2str(fbest_a)]);
    disp(['N. of Iterations x0: ', num2str(iter_a),'/',
       num2str(kmax)]);
    disp(['Rate of convergence: ', num2str(roc_a)]);
    disp(['Execution time x0_a: ', num2str(time_a), '
       seconds']);
```

```
36
37
38      t1_b = tic;
39      [xbest_b, iter_b, fbest_b, roc_b] = nelderMead(f,
            x0_b, rho, chi, gamma, sigma, kmax, tol,0);
40      time_b = toc(t1_b);
41
42      xbest_b = xbest_b';
43      roc_b = roc_b(end);
44      norm_gradf_b = norm(gradient(xbest_b));
45
46      disp('****Results for vector x0_b:****');
47      disp('Final x:'), disp(xbest_b);
48      disp(['Norma di gradf(xk): ', num2str(norm_gradf_b)
            ]);
49      disp(['f(xk): ', num2str(fbest_b)]);
50      disp(['N. of Iterations x0: ', num2str(iter_b),'/',
            num2str(kmax)]);
51      disp(['Rate of convergence: ', num2str(roc_b)]);
52
53      disp(['Execution time x0_b: ', num2str(time_b), '
            seconds']);
54
55
56      %avg of the results
57      fbest_avg = (fbest_b+fbest_a)/2;
58      iter_avg = (iter_b+iter_a)/2;
59      time_avg = (time_b+time_a)/2;
60      norm_gradf_avg = (norm_gradf_b+norm_gradf_a)/2;
61      roc_avg = (roc_a+roc_b )/ 2;
62
63      disp(['Avg f(xk): ', num2str(fbest_avg)]);
64      disp(['Avg Norma di gradf(xk): ', num2str(
            norm_gradf_avg)]);
65      disp(['Avg N. of Iterations x0: ', num2str(iter_avg)
            ,'/',num2str(kmax)]);
66      disp(['Execution time x0_b: ', num2str(time_avg), '
            seconds']);
67
68      xbest_avg = (xbest_b+xbest_a)/2;
69
70
71      T_a = table([xbest_a; xbest_b; xbest_avg], ...
```

```
72          [fbest_a; fbest_b; fbest_avg], ...
73          [iter_a; iter_b; iter_avg], ...
74          [time_a; time_b; time_avg], ...
75          [roc_a; roc_b; roc_avg], ...
76          'VariableNames', {'xbest', 'fbest', 'n_iter',
               'exec_time', 'roc'}, ...
77          'RowNames', {'x_a^(0)', 'x_b^(0)', 'Avg'});  %
               Add custom row names
78
79
80      % Display the table
81      disp(T_a);
82
83
84      %% MODIFIED NEWTON
85      %Newton parameters
86      rho = 0.5;
87      c1 = 1e-4;
88      btmax = 50;
89      tolgrad = 1e-6;
90      tau_kmax = 100;
91      kmax = 5000;
92
93
94      disp('****Newton analysis:****');
95
96      t1_a = tic;
97      [xbest_a, xseq_a, fk_a, norm_gradf_a, btseq_a] =
           modified_Newton_beta(f, g, h, x0_a, kmax, rho, c1
           , btmax, tolgrad, tau_kmax);
98      time_a = toc(t1_a);
99
100     xbest_a = xbest_a';
101     roc_a = rateOfConvergence([1 1], xseq_a);
102
103     disp('****Results for vector x0_a:****');
104     disp('Final x:'), disp(xbest_a);
105     disp(['Norma di gradf(xk): ', num2str(norm_gradf_a)
           ]);
106     disp(['f(xk): ', num2str(fk_a)]);
107     disp(['N. of Iterations x0_a: ', num2str(length(
           btseq_a)),'/',num2str(kmax)]);
```

```
108     disp(['Execution time x0_b: ', num2str(time_a), '
            seconds']);
109     disp(['Rate of convergence: ', num2str(roc_a)]);
110
111
112
113
114     t1_b = tic;
115     [xbest_b, xseq_b, fk_b, norm_gradf_b, btseq_b] =
            modified_Newton_beta(f, g, h, x0_b, kmax, rho, c1
            , btmax, tolgrad, tau_kmax);
116     time_b = toc(t1_b);
117
118     xbest_b = xbest_b';
119     roc_b = rateOfConvergence([1 1], xseq_b);
120
121     disp('****Results for vector x0_b:****');
122     disp('Final x:'), disp(xbest_b);
123     disp(['Norma di gradf(xk): ', num2str(norm_gradf_b)
            ]);
124     disp(['f(xk): ', num2str(fk_b)]);
125     disp(['N. of Iterations x0_b: ', num2str(length(
            btseq_b)),'/',num2str(kmax)]);
126     disp(['Execution time x0_b: ', num2str(time_b), '
            seconds']);
127     disp(['Rate of convergence: ', num2str(roc_b)]);
128
129
130
131
132     %avg of the results
133     xbest_avg = (xbest_b+xbest_a)/2;
134     fbest_avg = (fbest_b+fbest_a)/2;
135     iter_avg = (iter_b+iter_a)/2;
136     time_avg = (time_b+time_a)/2;
137     norm_gradf_avg = (norm_gradf_b+norm_gradf_a)/2;
138     roc_avg = (roc_a+roc_b )/ 2;
139
140     disp(['Avg f(xk): ', num2str(fbest_avg)]);
141     disp(['Avg Norma di gradf(xk): ', num2str(
            norm_gradf_avg)]);
142     disp(['Avg N. of Iterations x0: ', num2str(iter_avg)
            ,'/',num2str(kmax)]);
```

```
143     disp(['Execution time x0_b: ', num2str(time_avg), '
            seconds']);
144
145     disp(['Rate of convergence: ', num2str(roc_avg)]);
146
147     T_b = table([xbest_a; xbest_b; xbest_avg], ...
148           [fbest_a; fbest_b; fbest_avg], ...
149           [norm_gradf_a; norm_gradf_b; norm_gradf_avg],
                ...
150           [iter_a; iter_b; iter_avg], ...
151           [time_a; time_b; time_avg], ...
152           [roc_a; roc_b; roc_avg], ...
153           'VariableNames', {'xbest', 'fbest', 'grad_fk',
                'n_iter', 'exec_time', 'roc'}, ...
154           'RowNames', {'x_a^(0)', 'x_b^(0)', 'Avg'});
155
156
157     % Display the table
158     disp(T_b);
159
160
161 end
```

wrapperNelder.m

```
function wrapperNelder(dimensioni_nelder, problema,
   parameters)

    %Pick f dependign on problem
    switch problema
        case 1
            f = problema1();
        case 16
            f = problema16();
        case 25
            f = problema25();
        otherwise
            fprint("Errore");
    end

    [fbest1, iter1, time1, roc1, fail1] =
       wrapperNelderDim(dimensioni_nelder(1), problema,
       f, 1, 10, parameters);
    [fbest2, iter2, time2, roc2, fail2] =
       wrapperNelderDim(dimensioni_nelder(2), problema,
       f, 1, 10, parameters);
    [fbest3, iter3, time3, roc3, fail3] =
       wrapperNelderDim(dimensioni_nelder(3), problema,
       f, 1, 10, parameters);

    %We print our results in a table
     T = table([fbest1; fbest2; fbest3], ...
          [iter1; iter2; iter3], ...
          [time1; time2; time3], ...
          [roc1; roc2; roc3], ...
          [fail1; fail2; fail3],...
          'VariableNames', {'fbest','n_iter', 'exec_time
             ', 'roc', 'n_fails'}, ...
          'RowNames', {sprintf('N=%d', dimensioni_nelder
             (1)), ...
          sprintf('N=%d', dimensioni_nelder(2)), ...
          sprintf('N=%d', dimensioni_nelder(3))});

    display(T);
end
```

wrapperNelderDim.m

```
function [fbest_avg,iter_avg,time_avg,roc,n_failures] =
   wrapperNelderDim(dim, problema, f, attiva_newpoints,
   n_newpoints, parameters)
    x0 = zeros(dim,1);

    %Nelder mead parameters
    rho = parameters(1);
    chi = parameters(2);
    gamma = parameters(3);
    sigma = parameters(4);
    kmax = parameters(5);
    tol = parameters(6);

    %Taking x0 dependign on problem
    switch problema
        case 1
            x0(1:2:end) = -1.2;
            x0(2:2:end) = 1.0;
        case 13
            x0(1:2:end) = -1;
            x0(2:2:end) = 1;
        case 16
            x0(1:end) = 1;
        case 25
            x0(1:2:end) = -1.2;
            x0(2:2:end) = 1.0;
        otherwise
            fprint("Errore");
    end

    %Variables for avgs
    fbest_avg = 0;
    iter_avg = 0;
    time_avg = 0;
    roc = 0;
    n_failures = 0; %I consider fail when iter == kmax


    %x0
    t1 = tic;
```

```
39      [xbest, iter, fbest,roc_v] = nelderMead2(f, x0, rho,
            chi, gamma, sigma, kmax, tol, 0);
40      time = toc(t1);
41
42      %compute avgs
43      fbest_avg = fbest_avg + fbest;
44      iter_avg = iter_avg + iter;
45      if (iter >= kmax)
46          n_failures = n_failures+1;
47      end
48      time_avg = time_avg + time;
49      roc = roc + roc_v(end);
50
51      disp("****x0 results****");
52      disp("x0 xbest:"); disp(xbest);
53      printResultsNelder(fbest, iter, kmax, time, roc);
54
55      %pertubated starts, activated through parameter
56      if (attiva_newpoints)
57
58          disp("****Other results****");
59          %uniform distributed starting points
60          new_starting_points = generate_hypercube_points(
               x0, n_newpoints);
61
62          for i=1:10
63              x0 = new_starting_points(:, i);
64
65              t1 = tic;
66              [~, iter, fbest, roc_v] = nelderMead2(f, x0,
                    rho, chi, gamma, sigma, kmax, tol, 0);
67              time = toc(t1);
68
69              fbest_avg = fbest_avg + fbest;
70              iter_avg = iter_avg + iter;
71              if (iter >= kmax)
72                   n_failures = n_failures+1;
73              end
74              time_avg = time_avg + time;
75              roc = roc + roc_v(end);
76
77
```

```
78              printResultsNelder(fbest, iter, kmax, time,
                    roc);
79
80          end
81
82
83          %print avg
84          fbest_avg = fbest_avg/11;
85          iter_avg = iter_avg/11;
86          time_avg = time_avg/11;
87          roc = roc/11;
88
89          disp("****Avg results****");
90          printResultsNelder(fbest, iter, kmax, time, roc)
                ;
91
92
93      end
94
95
96  end
```

wrapperNewtonExact.m

```
function wrapperNewtonExact(dimensioni, problema,
   parameters)

    [fbest1, grad_norm1, iter1, time1, roc1, fail1] =
       wrapperNewtonExactDim(dimensioni(1), problema, 1,
        10, parameters);
    [fbest2, grad_norm2, iter2, time2, roc2, fail2] =
       wrapperNewtonExactDim(dimensioni(2), problema, 1,
        10, parameters);
    [fbest3, grad_norm3, iter3, time3, roc3, fail3] =
       wrapperNewtonExactDim(dimensioni(3), problema, 1,
        10, parameters);

    %Tabel
     T = table([fbest1; fbest2; fbest3], ...
          [grad_norm1; grad_norm2; grad_norm3], ...
          [iter1; iter2; iter3], ...
          [time1; time2; time3], ...
          [roc1; roc2; roc3], ...
          [fail1; fail2; fail3],...
          'VariableNames', {'fbest','grad_norm','n_iter'
             , 'exec_time', 'roc', 'n_fails'}, ...
          'RowNames', {sprintf('N=%d', dimensioni(1)),
             ...
          sprintf('N=%d', dimensioni(2)), ...
          sprintf('N=%d', dimensioni(3))});

    display(T);
end
```

wrapperNewtonExactDim.m

```
function [fbest_avg, grad_norm_avg, iter_avg, time_avg,
   roc_avg, n_fails] = wrapperNewtonExactDim(dim,
   problema, attiva_newpoints, n_newpoints, parameters)
    x0 = zeros(dim,1);

   %Newton parameters
    rho = parameters(1);
    c1 = parameters(2);
    btmax = parameters(3);
    tolgrad = parameters(4);
    tau_kmax = parameters(5);
    kmax = parameters(6);


    %Variables for avgs
    fbest_avg = 0;
    grad_norm_avg = 0;
    iter_avg = 0;
    time_avg = 0;
    roc_avg = 0;
    n_fails = 0;


    switch problema
        case 1
            x0(1:2:end) = -1.2;
            x0(2:2:end) = 1.0;
            [f,g,h] = problema1();
        case 13
            x0(1:2:end) = -1;
            x0(2:2:end) = 1;
            [f,g,h] = problema13();
        case 16
            x0(1:end) = 1;
            [f,g,h] = problema16();
        case 25
            x0(1:2:end) = -1.2;
            x0(2:2:end) = 1.0;
            [f,g,h] = problema25();

        otherwise
```

```
40              fprint("Errore");
41      end
42
43
44      %x0
45      t1 = tic;
46      [xbest, fbest, grad_norm, roc, iter] =
           modified_Newton_beta(f, g, h, x0, kmax, rho, c1,
           btmax, tolgrad, tau_kmax);
47      time = toc(t1);
48
49      disp("**** x0 best ****"); disp(xbest');
50      fbest_avg = fbest_avg + fbest;
51      grad_norm_avg = grad_norm_avg + grad_norm;
52      iter_avg = iter_avg + iter;
53      if (iter>=kmax)
54          n_fails = n_fails+1;
55      end
56      time_avg = time_avg + time;
57      roc_avg = roc_avg + roc(end);
58
59
60      printResults(grad_norm, fbest, iter, kmax, time, roc
           (end));
61
62      if (attiva_newpoints)
63          %uniform distributed starting points
64          new_starting_points = generate_hypercube_points(
               x0, n_newpoints);
65
66          for i=1:10
67              x0 = new_starting_points(:, i);
68
69              t1 = tic;
70              [~, fbest, grad_norm, roc, iter] =
                   modified_Newton_beta(f, g, h, x0, kmax,
                   rho, c1, btmax, tolgrad, tau_kmax);
71              time = toc(t1);
72
73              fbest_avg = fbest_avg + fbest;
74              grad_norm_avg = grad_norm_avg + grad_norm;
75              iter_avg = iter_avg + iter;
76              if (iter>=kmax)
```

```
77                  n_fails = n_fails+1;
78              end
79              time_avg = time_avg + time;
80              roc_avg = roc_avg + roc(end);
81
82
83              printResults(grad_norm, fbest, iter, kmax,
                    time, roc(end));
84
85
86          end
87
88
89          %print avg
90          fbest_avg = fbest_avg/11;
91          grad_norm_avg = grad_norm_avg/11;
92          iter_avg = iter_avg/11;
93          time_avg = time_avg/11;
94          roc_avg = roc_avg/11;
95
96          disp("*****AVG*****");
97          printResults(grad_norm_avg, fbest_avg, iter_avg,
                kmax, time_avg, roc_avg);
98
99      end
100
101
102 end
```

wrapperNewtonFinDiff.m

```
function wrapperNewtonFinDiff(dim, problema, parameters,
    type_h)


    e = [2,4,6,8,10,12];
    h = zeros(6,1);
    for i=1:6
        h(i) = 10^(-e(i));
    end

    [fbest1, grad_norm1, iter1, time1, roc1, fail1] =
       wrapperNewtonFinDiffDim(dim, problema, 1, 10,
       parameters, type_h, h(1));
    [fbest2, grad_norm2, iter2, time2, roc2, fail2] =
       wrapperNewtonFinDiffDim(dim, problema, 1, 10,
       parameters, type_h, h(2));
    [fbest3, grad_norm3, iter3, time3, roc3, fail3] =
       wrapperNewtonFinDiffDim(dim, problema, 1, 10,
       parameters, type_h, h(3));
    [fbest4, grad_norm4, iter4, time4, roc4, fail4] =
       wrapperNewtonFinDiffDim(dim, problema, 1, 10,
       parameters, type_h, h(4));
    [fbest5, grad_norm5, iter5, time5, roc5, fail5] =
       wrapperNewtonFinDiffDim(dim, problema, 1, 10,
       parameters, type_h, h(5));
    [fbest6, grad_norm6, iter6, time6, roc6, fail6] =
       wrapperNewtonFinDiffDim(dim, problema, 1, 10,
       parameters, type_h, h(6));

    %Tabella
    disp(["***Risultati per dimensione ", num2str(dim),"
        e caso ", type_h]);
     T = table([fbest1; fbest2; fbest3; fbest4; fbest5;
        fbest6], ...
          [grad_norm1; grad_norm2; grad_norm3;grad_norm4
             ; grad_norm5; grad_norm6], ...
          [iter1; iter2; iter3; iter4; iter5; iter6],
             ...
          [time1; time2; time3; time4;time5;time6], ...
          [roc1; roc2; roc3;roc4;roc5;roc6], ...
          [fail1; fail2; fail3;fail4;fail5;fail6],...
```

```
25            'VariableNames', {'fbest','grad_norm','n_iter'
                 , 'exec_time', 'roc', 'n_fails'}, ...
26            'RowNames', {sprintf('h=%d', h(1)), ...
27            sprintf('h=%d', h(2)), ...
28            sprintf('h=%d', h(3)), ...
29            sprintf('h=%d', h(4)) ...
30            sprintf('h=%d', h(5)) ...
31            sprintf('h=%d', h(6))});
32
33        display(T);
34    end
```

wrapperNewtonFinDiffDim.m

```
function [fbest_avg, grad_norm_avg, iter_avg, time_avg,
   roc_avg, n_fails] = wrapperNewtonFinDiffDim(dim,
   problema, attiva_newpoints, n_newpoints, parameters,
   type_h, step)
    x0 = zeros(dim,1);

   %Newton parameters
    rho = parameters(1);
    c1 = parameters(2);
    btmax = parameters(3);
    tolgrad = parameters(4);
    tau_kmax = parameters(5);
    kmax = parameters(6);


    %Variables for avgs
    fbest_avg = 0;
    grad_norm_avg = 0;
    iter_avg = 0;
    time_avg = 0;
    roc_avg = 0;
    n_fails = 0;


    switch problema
        case 1
            x0(1:2:end) = -1.2;
            x0(2:2:end) = 1.0;
            f = problema1();
            [g,h] = finite_diff_handles_1(type_h, step);
        case 16
            x0(1:end) = 1;
            f = problema16();
            [g,h] = finite_diff_handles_16(type_h, step)
               ;
        case 25
            x0(1:2:end) = -1.2;
            x0(2:2:end) = 1.0;
            f = problema25();
            [g,h] = finite_diff_handles_25(type_h, step)
               ;
```

```
        otherwise
            fprint("Errore");
    end


    %x0
    t1 = tic;
    [xbest, fbest, grad_norm, roc, iter] =
       modified_Newton_beta(f, g, h, x0, kmax, rho, c1,
       btmax, tolgrad, tau_kmax);
    time = toc(t1);

    disp("**** x0 best ****");% disp(xbest');
    fbest_avg = fbest_avg + fbest;
    grad_norm_avg = grad_norm_avg + grad_norm;
    iter_avg = iter_avg + iter;
    if (iter>=kmax)
        n_fails = n_fails+1;
    end
    time_avg = time_avg + time;
    roc_avg = roc_avg + roc(end);


    printResults(grad_norm, fbest, iter, kmax, time, roc
       (end));

    if (attiva_newpoints)

        new_starting_points = generate_hypercube_points(
           x0, n_newpoints);

        for i=1:10
            x0 = new_starting_points(:, i);

            t1 = tic;
            [~, fbest, grad_norm, roc, iter] =
               modified_Newton_beta(f, g, h, x0, kmax,
               rho, c1, btmax, tolgrad, tau_kmax);
            time = toc(t1);

            fbest_avg = fbest_avg + fbest;
            grad_norm_avg = grad_norm_avg + grad_norm;
            iter_avg = iter_avg + iter;
```

```
74              if (iter>=kmax)
75                  n_fails = n_fails+1;
76              end
77              time_avg = time_avg + time;
78              roc_avg = roc_avg + roc(end);
79
80
81              printResults(grad_norm, fbest, iter, kmax,
                    time, roc(end));
82
83
84          end
85
86
87          %print avg
88          fbest_avg = fbest_avg/11;
89          grad_norm_avg = grad_norm_avg/11;
90          iter_avg = iter_avg/11;
91          time_avg = time_avg/11;
92          roc_avg = roc_avg/11;
93
94          disp("*****AVG*****");
95          printResults(grad_norm_avg, fbest_avg, iter_avg,
                kmax, time_avg, roc_avg);
96
97      end
98
99
100 end
```

printResults.m

```
function printResults(grad_norm, fbest, iter, kmax, time
    , roc)

    disp(['f(xk): ', num2str(fbest)]);
    disp(['norm gradf(xk): ', num2str(grad_norm)]);
    disp([' N. of Iterations : ', num2str(iter),'/',
       num2str(kmax)]);
    disp(['Execution time: ', num2str(time), ' seconds'
       ]);
    disp(['Roc: ', num2str(roc)]);

end
```

printResultsNelder.m

```
function printResultsNelder(fbest, iter, kmax, time, roc
    )

    disp(['f(xk): ', num2str(fbest)]);
    disp([' N. of Iterations : ', num2str(iter),'/',
        num2str(kmax)]);
    disp(['Execution time: ', num2str(time), ' seconds'
        ]);
    disp(['Roc: ', num2str(roc)]);

end
```

nelderSorter.m

```
function  [fk_sorted, indices] = nelderSorter(simplex,n,
   f)

    fk = zeros(n+1,1);
    for i = 1:n+1
        fk(i) = f(simplex(:, i));
    end
    [fk_sorted, indices] = sort(fk);

end
```

nelderMead2.m

```
function [xbest, iter, fbest, roc] = nelderMead2(f, x0,
    rho, chi, gamma, sigma, kmax, tol, enablePlotting)
    % The function implements the Nelder-Mead algorithm
       for optimizing an objective function f.
    % Inputs:
    %   - f: Objective function to minimize.
    %   - x0: Initial guess (starting point).
    %   - rho, chi, gamma, sigma: Algorithm parameters
       for reflection, expansion, contraction, and
       shrinkage.
    %   - kmax: Maximum number of iterations.
    %   - tol: Convergence tolerance (when to stop).
    %   - enablePlotting: Flag to enable or disable
       plotting of convergence.
    % Outputs:
    %   - xbest: Best solution found (final simplex
       point).
    %   - iter: Number of iterations performed.
    %   - fbest: Objective function value at the best
       solution.
    %   - roc: Rate of convergence (log-based).

    n = length(x0);  % The number of variables (
       dimensions).

    % Initialize the simplex matrix (n by n+1) where
       each column is a point.
    simplex = zeros(n, n+1);
    simplex(:,1) = x0;  % The first point in the simplex
        is the starting point.

    % Generate the other n points of the simplex by
       perturbing the starting point.
    for i = 1:n
        ei = zeros(n,1);  % Create a unit vector.
        ei(i) = 0.05 * abs(x0(i));  % Perturb in the i-
           th direction.
        simplex(:, i+1) = x0 + ei;  % Add the perturbed
           point to the simplex.
    end

```

```
29      % Sort the simplex points based on their function
            values.
30      [fk_sorted, indices] = nelderSorter(simplex, n, f);
31      x_best = simplex(:, indices(1));  % The best point
            is the one with the lowest function value.
32      x_history = x_best;  % Track the history of the best
            solutions for ROC calculation.
33
34      roc = [];  % Initialize Rate of Convergence (ROC)
            array.
35      iter = 0;  % Start iteration counter.
36
37      % Iterate until maximum iterations or convergence
            tolerance is met.
38      while iter < kmax && (fk_sorted(n) - fk_sorted(1)) >
            tol
39          iter = iter + 1;  % Increment iteration counter.
40
41          % Compute the centroid (mean) of the n best
                points (exclude the worst).
42          centroid = mean(simplex(:, indices(1:end-1)), 2)
                ;
43
44          % Reflection step: Reflect the worst point
                across the centroid.
45          xR = centroid + rho * (centroid - simplex(:,
                indices(end)));
46          fxR = f(xR);  % Function value at the reflected
                point.
47
48          % Check different conditions to decide the next
                move.
49          if fxR >= fk_sorted(1) && fxR < fk_sorted(n)
50              % If the reflected point is better than the
                    worst but not the best.
51              simplex(:, indices(end)) = xR;
52          elseif fxR < fk_sorted(1)  % If reflection gives
                the best point.
53              % Expansion step: Try to expand the
                    reflected point further.
54              xE = centroid + chi * (xR - centroid);
55              if f(xE) < fxR
56                  simplex(:, indices(end)) = xE;
```

```
57              else
58                  simplex(:, indices(end)) = xR;  % Keep
                        reflected point if expansion is worse
                        .
59              end
60          else  % If the reflected point is worse than the
                worst.
61              % Contraction step: Try contracting the
                    worst point.
62              if fxR > fk_sorted(n)
63                  xC = centroid + gamma * (simplex(:,
                        indices(end)) - centroid);
64              else
65                  xC = centroid + gamma * (xR - centroid);
66              end
67              if f(xC) < fk_sorted(end)
68                  simplex(:, indices(end)) = xC;
69              else  % Shrink the simplex if contraction is
                        worse.
70                  for i = 2:n+1
71                      simplex(:, i) = simplex(:, indices
                            (1)) + sigma * (simplex(:, i) -
                            simplex(:, indices(1)));
72                  end
73              end
74          end
75
76          % Re-sort the simplex points after the
                modification.
77          [fk_sorted, indices] = nelderSorter(simplex, n,
                f);
78          x_best = simplex(:, indices(1));  % Update best
                solution.
79          x_history = [x_history, x_best];  % Track the
                history of best solutions.
80
81          % Calculate Rate of Convergence (ROC) after at
                least 2 iterations.
82          if iter > 2
83              norm1 = norm(x_history(:, end) - x_history
                    (:, end-1));  % Difference between
                    current and previous best.
```

```
84              norm2 = norm(x_history(:, end-1) - x_history
                    (:, end-2));  % Difference between
                    previous and second-to-last best.
85
86              if norm2 > 0
87                  roc(end+1) = log(norm1) / log(norm2);  %
                        Logarithmic rate of convergence.
88              end
89          end
90
91          % Plot the convergence every 10 iterations if
                enabled.
92          if enablePlotting && mod(iter, 10) == 0
93              figure(1);
94              plot(vecnorm(x_history - x_history(:,end),
                    2, 1), '-o', 'MarkerSize', 4, 'LineWidth'
                    , 1.5);
95              xlabel('Iterations');
96              ylabel('Norm Difference');
97              title('Nelder-Mead Convergence (Vector-Based
                    )');
98              grid on;
99              drawnow;
100         end
101     end
102
103     % Output the best solution and function value.
104     xbest = simplex(:, indices(1));  % Final best point.
105     fbest = fk_sorted(1);  % Function value at the best
            point.
106 end
```

modifiedNewtonBeta.m

```
function [xk, fk, gradfk_norm, roc, k] = ...
    modified_Newton_beta(f, gradf, Hessf, x, kmax, rho,
       c1, btmax, tolgrad, tau_kmax)

    % Helper function for the Armijo condition in the
       line search.
    farmijo = @(fk, alpha, c1_gradfk_pk) fk + alpha *
       c1_gradfk_pk;

    % Initialization
    n = length(x);  % Length of the input (dimension of
       the problem)
    xk = x;          % Initialize the current point as
       the starting point
    fk = f(xk);      % Compute the function value at the
        starting point
    gradfk = gradf(xk);   % Compute the gradient at the
       starting point
    gradfk_norm = norm(gradfk);  % Compute the norm of
       the gradient
    Hessfk = Hessf(xk);       % Compute the Hessian at
       the starting point
    k = 0;           % Initialize the iteration counter

    roc = [];        % Initialize the rate of
       convergence (empty at first)
    x_prev = xk;     % Store the initial point for
       convergence tracking

    % Main optimization loop
    while k < kmax && gradfk_norm >= tolgrad

        % Regularization parameter to modify Hessian if
           necessary
        beta = 1e-3;  % Small constant for
           regularization
        min_diag = min(diag(Hessfk));  % Find the
           minimum diagonal element of the Hessian

        % If the minimum diagonal element is positive,
           no regularization needed
```

```
        if min_diag > 0
            tau = 0;
        else
            tau = -min_diag + beta;  % Regularize
               Hessian if necessary
        end

        % Adjust the regularization parameter tau to
           make Hessian positive definite
        for k_tau = 1:tau_kmax
            Bk = Hessfk + tau * speye(n);  % Add
               regularization term to Hessian
            [R, p] = chol(Bk);  % Cholesky decomposition
                of the regularized Hessian
            if p == 0, break; end  % If successful,
               break the loop
            tau = max(beta, 10 * tau);  % Increase tau
               if Cholesky fails
        end

        % Compute the Newton direction (using Cholesky
           factorization)
        y = -R' \ gradfk;  % Solve the linear system for
            y
        pk = R \ y;        % Solve the linear system for
            pk (the Newton direction)

        % Backtracking line search to find the optimal
           step size alpha
        alpha = 1;  % Initial step size
        c1_gradfk_pk = c1 * (gradfk' * pk);  % Compute
           c1 * (gradfk' * pk) for Armijo condition
        bt = 0;  % Initialize backtracking counter

        % Perform backtracking line search
        while bt < btmax && f(xk + alpha * pk) > farmijo
           (fk, alpha, c1_gradfk_pk)
            alpha = rho * alpha;  % Decrease alpha by a
               factor of rho
            bt = bt + 1;  % Increment backtracking
               counter
        end

```

```
56          % If maximum backtracking iterations are reached
                and condition is not satisfied, break
57          if bt == btmax && f(xk + alpha * pk) > farmijo(
                fk, alpha, c1_gradfk_pk)
58              break;
59          end
60
61          % Update the solution with the step size alpha
62          xnew = xk + alpha * pk;
63          fk = f(xnew);  % Compute the function value at
                the new point
64          gradfk = gradf(xnew);  % Compute the gradient at
                the new point
65          gradfk_norm = norm(gradfk);  % Compute the norm
                of the new gradient
66          Hessfk = Hessf(xnew);  % Compute the Hessian at
                the new point
67
68          x_prev = [x_prev, xnew];  % Store the current
                solution for rate of convergence computation
69
70          % Rate of convergence (ROC) computation
71          if k > 1
72              norm1 = norm(x_prev(:, end) - x_prev(:, end
                    -1));  % Compute the difference between
                    the last two points
73              norm2 = norm(x_prev(:, end-1) - x_prev(:,
                    end-2));  % Compute the difference
                    between the previous two points
74              if norm2 > 0
75                  roc(end+1) = log(norm1) / log(norm2);  %
                        Logarithmic rate of convergence
76              end
77          end
78
79          % Update the current point and iteration counter
80          xk = xnew;  % Update current point
81          k = k + 1;  % Increment iteration counter
82      end
83  end
```

finiteDiffHandles1.m

```
function [grad_handle, hess_handle] =
   finite_diff_handles_1(type_h, step)

    grad_handle = @(x) compute_gradient(x, step, type_h)
       ;
    hess_handle = @(x) compute_hessian(x, step, type_h);
end


function grad_approx = compute_gradient(x, step, type_h)
    n = length(x);
    grad_approx = zeros(n,1);
    h = step * ones(n,1);

    if type_h == 1
        h = step * abs(x);
    end


    grad_approx(1) = 400*x(1)*(x(1)^2 + h(1)^2 - x(2)) +
        2*(x(1)-1);


    for i = 2:n
        grad_approx(i) = (1 / (2 * h(i))) * ...
            (100 * (((x(i-1) + h(i-1))^2 - (x(i) + h(i))
               )^2 - ...
            ((x(i-1) - h(i-1))^2 - (x(i) - h(i)))^2) +
               ...
            (((x(i-1) + h(i-1)) - 1)^2 - ((x(i-1) - h(i
               -1)) - 1)^2));
    end
end


function hess_approx = compute_hessian(x, step, type_h)
    n = length(x);
    hess_approx = sparse(n,n);
    h = step * ones(n,1);

```

```
36      if type_h == 1
37          h = step * abs(x);
38      end
39
40      %main diag
41      hess_approx(1,1) = (100*(x(1) + 2*h(1))^4 - 1600*(h
           (1)^2*x(2)) +8*h(1)^2 -200*(x(1))^4 + 100*(x(1)
           -2*h(1))^2)/(h(1)^2);
42      hess_approx(n,n) = 800*h(n)^2 - x(n-1)^2 + 2*x(n-1)
           +2*x(n)^2 - 2*x(n);
43      for i=2:n-1
44          hess_approx(i,i) = (800*h(i) - x(i-1)^2 + 2*x(i
               -1) + x(i)^2 -2*x(i) +100*(x(i)+2*h(i))^4 ...
45          -1600*(h(i)^2)*x(i+1) + 8*h(i)^2 - 200*x(i)^4
               +100*(x(i) - 2*h(i))^2)/(h(i)^2);
46      end
47
48
49      %upper diag
50      hess_approx(1,2) = (-1600*h(1)*h(2)*x(1))/(4*h(1)^2)
           ;
51      hess_approx(n-1, n) = (-400*x(n-1)*h(n-1))/h(n);
52
53      for i=2:n-1
54          hess_approx(i,i+1) = (-400*x(i)*h(i+1))/h(i);
55      end
56
57      hess_approx= max(hess_approx, hess_approx');
58  end
```

finiteDiffHandles16.m

```
function [grad_handle, hess_handle] =
   finite_diff_handles_16(type_h,h)


    grad_handle = @(x) compute_gradient(x, h, type_h);
    hess_handle = @(x) compute_hessian(x, h, type_h);
end

function grad_approx = compute_gradient(x, step, type_h)
    n = length(x);
    grad_approx = zeros(n,1);
    h = step * ones(n,1);
    if type_h == 1
        h = step * abs(x);
    end

    for i = 1:n-1
        grad_approx(i) = (2*i*sin(x(i))*sin(h(i)) + 4*
           cos(x(i))*sin(h(i))) / (2*h(i));
    end

    grad_approx(n) = (2*n*sin(x(n))*sin(h(n)) - 2*(n-1)*
       cos(x(n))*sin(h(n))) / (2*h(n));
end

function Hess_approx = compute_hessian(x, step, type_h)
    n = length(x);
    Hess_approx = sparse(n,n);

    h = step * ones(n,1);
    if type_h == 1
        h = step * abs(x);
    end

    for i = 1:n-1
        Hess_approx(i,i) = (i*cos(x(i)) - 2*sin(x(i))) -
            h(i)*(i*sin(x(i)) + 2*cos(x(i)));
    end

    Hess_approx(n,n) = (n*cos(x(n)) - (n-1)*sin(x(n))) -
        h(n)*(n*sin(x(n)) - (n-1)*cos(x(n)));
```

```
37 end
```

finiteDiffHandles25.m

```
function [grad_handle, hess_handle] =
   finite_diff_handles_25(type_h,h)

    grad_handle = @(x) findiff_grad_25(x, h, type_h);

    hess_handle = @(x) findiff_hess_25(x, h, type_h);
end

function grad_approx = findiff_grad_25(x, h, type_h)
    n = length(x);
    grad_approx = zeros(n, 1);

    if mod(n, 2) == 0

        for k = 1:n
            switch type_h
                case 1
                    passok = h * abs(x(k));
                case 0
                    passok = h;
            end

            if mod(k, 2) == 0
                grad_approx(k, 1) = (-40 * passok * (10
                   * x(k - 1)^2 - 10 * x(k))) / (4 *
                   passok);
            else
                grad_approx(k, 1) = (80 * x(k) * passok
                   * (10 * x(k)^2 + 10 * passok^2 - 10 *
                    x(k + 1)) + 4 * passok * (x(k) - 1))
                    / (4 * passok);
            end
        end
    else

        for k = 1:n
            switch type_h
                case 1
                    passok = h * abs(x(k));
                case 0
                    passok = h;
```

```
            end

            if k == 1
                grad_approx(1, 1) = (80 * x(k) * passok
                    * (10 * x(k)^2 + 10 * passok^2 - 10 *
                     x(k + 1)) + 4 * passok * (x(k) - 1)
                    - 40 * passok * (10 * x(n)^2 - 10 * x
                    (1))) / (4 * passok);
            elseif k == n
                grad_approx(k, 1) = (80 * x(k) * passok
                    * (10 * x(k)^2 + 10 * passok^2 - 10 *
                     x(1))) / (4 * passok);
            elseif mod(k, 2) == 0
                grad_approx(k, 1) = (-40 * passok * (10
                    * x(k - 1)^2 - 10 * x(k))) / (4 *
                    passok);
            else
                grad_approx(k, 1) = (80 * x(k) * passok
                    * (10 * x(k)^2 + 10 * passok^2 - 10 *
                     x(k + 1)) + 4 * passok * (x(k) - 1))
                     / (4 * passok);
            end
        end
    end
end

function hessian_approx = findiff_hess_25(x, h, type_h)

    n = length(x);


    i_indices = [];
    j_indices = [];
    values = [];
    cont = 1;


    for k = 1:n

        if k == 1 && mod(n, 2) == 1

            switch type_h
                case 0
```

```
68                      hk = h;
69                  case 1
70                      hk = h * abs(x(k));
71              end
72
73              H_kk = (40 * hk^2 * (10 * x(k)^2 - 10 * x(k
                    + 1)) + 1400 * hk^4 + 2400 * hk^3 * x(k)
                    + 800 * x(k)^2 * hk^2 + 2 * hk^2 + 200 *
                    hk^2) / (2 * hk^2);
74              i_indices(cont) = k;
75              j_indices(cont) = k;
76              values(cont) = H_kk;
77              cont = cont + 1;
78
79          elseif k == n && mod(n, 2) == 1
80
81
82              switch type_h
83                  case 0
84                      hk = h;
85                  case 1
86                      hk = h * abs(x(k));
87              end
88
89              H_kk = (40 * hk^2 * (10 * x(k)^2 - 10 * x(1)
                    ) + 1400 * hk^4 + 2400 * hk^3 * x(k) +
                    800 * x(k)^2 * hk^2) / (2 * hk^2);
90              i_indices(cont) = k;
91              j_indices(cont) = k;
92              values(cont) = H_kk;
93              cont = cont + 1;
94
95          elseif mod(k, 2) == 1
96
97
98              switch type_h
99                  case 0
100                     hk = h;
101                 case 1
102                     hk = h * abs(x(k));
103             end
104
```

```
105             H_kk = (40 * hk^2 * (10 * x(k)^2 - 10 * x(k
                    + 1)) + 1400 * hk^4 + 2400 * hk^3 * x(k)
                    + 800 * x(k)^2 * hk^2 + 2 * hk^2) / (2 *
                    hk^2);
106             i_indices(cont) = k;
107             j_indices(cont) = k;
108             values(cont) = H_kk;
109             cont = cont + 1;
110
111         elseif mod(k, 2) == 0
112
113
114             switch type_h
115                 case 0
116                     hk = h;
117                 case 1
118                     hk = h * abs(x(k));
119             end
120
121             H_kk = (200 * hk^2) / (2 * hk^2);
122             i_indices(cont) = k;
123             j_indices(cont) = k;
124             values(cont) = H_kk;
125             cont = cont + 1;
126         end
127
128
129         if mod(n, 2) == 1 && k == n
130
131
132             switch type_h
133                 case 0
134                     h1 = h;
135                 case 1
136                     h1 = h * abs(x(1));
137             end
138
139             H_n1 = (20 * h1 * (-20 * x(k) * hk - 10 * hk
                    ^2)) / (2 * hk * h1);
140             i_indices(cont) = n;
141             j_indices(cont) = 1;
142             values(cont) = H_n1;
143             cont = cont + 1;
```

```
144
145             %Symmetry
146             i_indices(cont) = 1;
147             j_indices(cont) = n;
148             values(cont) = H_n1;
149             cont = cont + 1;
150
151         elseif mod(k, 2) == 1 && k < n
152
153             switch type_h
154                 case 0
155                     hk1 = h;
156                 case 1
157                     hk1 = h * abs(x(k + 1));
158             end
159
160             H_k_k1 = (20 * hk1 * (-10 * hk^2 - 20 * hk *
                    x(k))) / (2 * hk * hk1);
161             i_indices(cont) = k;
162             j_indices(cont) = k + 1;
163             values(cont) = H_k_k1;
164             cont = cont + 1;
165
166
167             i_indices(cont) = k + 1;
168             j_indices(cont) = k;
169             values(cont) = H_k_k1;
170             cont = cont + 1;
171         end
172     end
173
174     hessian_approx = sparse(i_indices, j_indices, values
            , n, n);
175 end
```